Proactive Spatial Routing for Implicit Neural Representations

Patrick Schrems | B.S. Senior in Computer Science & Applied Mathematics, Michigan State University
Contact: schremsp@msu.edu | *Repository:* `https://github.com/ptschrems/INRProactiveSpeedUp`

---

1. Motivation & The Computational Bottleneck

Implicit Neural Representations (INRs, e.g., SIRENs) parameterize continuous visual signals as coordinate networks $f_\theta : (x, y) \to (R, G, B)$. To scale to high-frequency visual content across large domains (here, large images), standard architectures partition space using hierarchical quadtrees.

However, existing frameworks operate **strictly reactively**: they optimize candidate INRs on large spatial patches for hundreds or thousands of gradient iterations, observe residual reconstruction error, and only then split if fitting fails. This trial-and-error cycle wastes unrecoverable compute time (t_S) on doomed parent nodes that were structurally incapable of fitting the local signal within the step budget.

2. Decision-Theoretic Cost Formulation

I propose an upfront **proactive oracle** $\phi(I) \to \hat{y} \in \{0, 1\}$ that evaluates raw patch features in negligible time ($t_o \ll t_S$) to predict whether local SIREN training will hit error threshold ϵ, bypassing training on complex blocks prior to gradient descent.

For an individual image chunk I with ground truth fittability $y(I) \in \{0, 1\}$ (0 = fittable, 1 = unfittable) and oracle decision $\hat{y} \in \{0, 1\}$ (0 = Train, 1 = Split), the runtime is modeled by the recurrence:

$$C(I) = t_o + \hat{y} \cdot 4C(I_{\text{child}}) + (1 - \hat{y})\Big(t_S + y(I) \cdot 4C(I_{\text{child}})\Big)$$

Taking expectations across tree depth d under natural patch smoothness prior $p_0(d) = \mathbb{P}(y = 0)$, Type I False Positive rate $\alpha_d = \mathbb{P}(\hat{y} = 1 \mid y = 0)$, and Type II False Negative rate $\beta_d = \mathbb{P}(\hat{y} = 0 \mid y = 1)$:

$$C(d) = t_o(d) + \underbrace{\Big[p_0(d)(1 - \alpha_d) + (1 - p_0(d))\beta_d\Big]t_S(d)}_{\text{Expected Local Training Time}} + \underbrace{4\Big[1 - p_0(d)(1 - \alpha_d)\Big]C(d + 1)}_{\text{Expected Downstream Subtree Time}}$$

with terminal leaf condition at maximum depth D: $C(D) = t_o(D) + t_S(D)$.

3. Theoretical Error Asymmetry & Empirical Validation

Evaluating the partial derivatives of $C(d)$ reveals a fundamental operational asymmetry:

- **False Negatives (β_d, Missed Splits):** Incur only a localized, additive training penalty: $\frac{\partial C(d)}{\partial \beta_d} = (1 - p_0(d))t_S(d)$. This penalty can be amortized near zero by transferring parent network weights as a warm-start initialization to child quadrants ($e_{\text{warm}} \ll e_{\text{fresh}}$).
- **False Positives (α_d, False Splits):** Incur an exponential subtree penalty: $\frac{\partial C(d)}{\partial \alpha_d} = p_0(d)[4C(d+1) - t_S(d)]$. Creating unnecessary children multiplies the effective branching factor by 4, causing catastrophic node explosion if α_d is unconstrained.
- **Empirical Hardware Proof-of-Concept:** An initial PyTorch prototype executed on an AMD discrete GPU via ROCm demonstrated that bypassing doomed root-level blocks reduced total wall-clock encoding time from **46.30 s down to 28.97 s (a 37.44% speedup)** while achieving target visual fidelity.

4. Open Pattern Recognition Questions to Finish

1. **Feature Space Design:** What lightweight visual feature representation $\mathbf{x} \in \mathbb{R}^k$ (e.g., directional gradient anisotropy, Total Variation, or 2D Fourier/wavelet spectral decay) best separates smooth from complex patches under the strict latency budget $t_o \ll t_S$?
2. **Decision Calibration Under Asymmetric Risk:** How should we train and calibrate the classifier (e.g., Neyman–Pearson classification with a hard constraint on Type I error $\alpha_d \leq \alpha_{\max}$, or asymmetric cost-sensitive loss) to enforce near-perfect specificity on shallow, high-risk nodes?